# Feature Extraction, Fine-Tuning

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



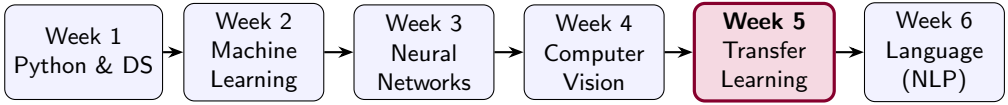

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 5, Day 30, Lecture

18 October 2026

# Table of Contents


1. Big Picture and Motivation
2. Learning Outcomes
3. Feature Extraction and Fine-Tuning: What Moves
4. Layer Freezing: How Much of the Encoder Moves
5. The Small-Data Regime and Catastrophic Forgetting
6. Olympiad Practice
7. Limitations and What Comes Next
8. Summary
9. References

# Where We Are




**Today: Day 30 of 42**

- **Yesterday (Day 29):** a pretrained ResNet-18 is an **encoder** (11,176,512 weights learned on ImageNet) and a **head** (one `Linear`). We froze the encoder, replaced the head by `Linear(512, 5)` and trained its 2,565 weights on the flowers: transfer learning, the frozen way.
- **Today:** the second way, **fine-tuning**: let part of the encoder move too. Which part (**layer freezing**), how fast (one learning rate per group), what to do with few photos (the **small-data regime**), and how to keep what the encoder knows (**catastrophic forgetting**).
- **This week:** Day 31 embeddings, similarity and CLIP; Day 32 today's choices step by step; Day 33 linear probing and CLIP zero-shot; Day 34 the small-data habits that win points; Friday the competition.

# How We Did It Before: The Encoder Frozen, the Head Trained

```
model = resnet18(weights="IMAGENET1K_V1")
for param in model.parameters():
    param.requires_grad = False    # nothing
        moves
model.fc = nn.Linear(512, 5)       # new head
optimizer = torch.optim.Adam(
    model.fc.parameters(), lr=1e-3)
```

- Yesterday's recipe: **2,565** of 11,179,077 weights moved. Lab 2 trained it in five epochs on 3,453 flower photos.
- The encoder's 512 numbers already keep daisies away from tulips; the head only draws lines between them.

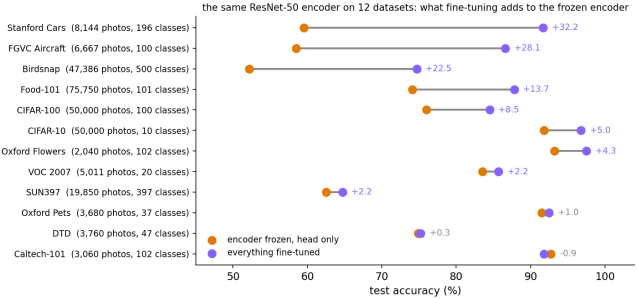
| Encoder frozen, head trained | photos | accuracy |
|---|---|---|
| Oxford Flowers, ResNet-50 | 2,040 | 93.2% |
| Caltech-101, ResNet-50 | 3,060 | 92.7% |
| Oxford Pets, ResNet-50 | 3,680 | 91.5% |
| ants against bees, ResNet-18 | 244 | 94.8% in 28 s |

Kornblith, Shlens and Le (2019), Appendix Table I.1, "logistic regression" column; the PyTorch transfer-learning tutorial's printed result.

## Why it works there

Flowers, pets and Caltech's objects are the kind of photo ImageNet is made of. The features the encoder learned already fit them, so a frozen encoder plus one `Linear` gets within a few points of the best.

# Where It Breaks: Photos the Encoder Was Not Made For




Source: Kornblith, Shlens and Le (2019), Appendix Table I.1, ResNet-50 rows; `scripts/day30_finetune_figures.py`. No model was trained for this deck.

## The problem

Stanford Cars: the frozen encoder gives **59.5%**, the same encoder fine-tuned **91.7%**: 32 points. Aircraft 28 points, bird species 22. Telling 196 car models apart needs features ImageNet's 1,000 classes never asked for. If two car models land on the same 512 numbers, no head can separate them: the *encoder* has to change.

# Today's Idea

### In one sentence

Keep the pretrained weights as the **starting point**, but let the stages that need to change **move**, with steps small enough to keep what they know. That is **fine-tuning**; choosing which stages move is **layer freezing**.

- **Feature extraction** (the encoder frozen, also called **linear probing**) against **fine-tuning** (the encoder moves): what each costs and wins.
- **One learning rate per parameter group**, the pretrained stages slower than the new head; **discriminative learning rates**, **gradual unfreezing**.
- **The small-data regime:** fewer photos per class, fewer weights move. **Catastrophic forgetting:** too much moved too fast.

### Where you will meet it

NEOAI 2025, *Underfitting CV*: a 102-class classifier handed over as a weights file, extra photos for its 10 weak classes only; "make sure it doesn't forget what it already knows" (today's practice task). Today's labs: the three strategies on the flowers; then brain scans, far from ImageNet.

# ⇒ Learning Outcomes

By the end of this session, you will be able to:

- **Explain** feature extraction (linear probing) against fine-tuning: which weights move, which stay, and what each costs in time and photos.
- **Explain** with the update rule and real numbers why the pretrained stages get a small learning rate and the new head a normal one.
- **Implement** layer freezing in PyTorch: freeze everything, thaw `layer4`, give each parameter group its own learning rate, unfreeze gradually.
- **Choose** how much of the encoder to unfreeze from the published evidence: distance from ImageNet, photos per class, time.
- **Recognise** the small-data regime and catastrophic forgetting, and apply the habits that keep the encoder's knowledge.
- **Recognise** a fine-tuning task in an Olympiad statement and plan the solution: probe, read, thaw, read again.

# Feature Extraction and Fine-Tuning: **What Moves**

# Two Definitions, and a Name to Learn

### Feature extraction

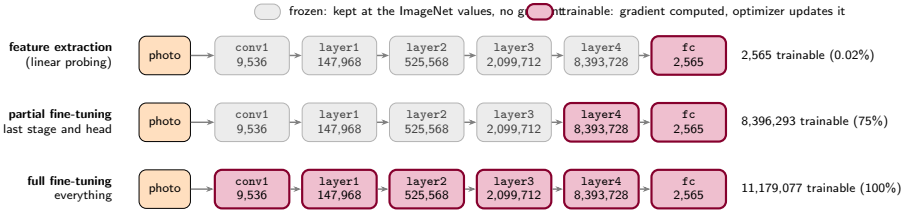
The encoder is **frozen** and used as a fixed function: photo $\rightarrow$ 512 numbers, the **features**. Only a new head is trained on those numbers. When the head is one `Linear` (a logistic regression on the features) the method is called **linear probing**: the probe *measures* how good the frozen features are for your classes.

### Fine-tuning

Start from the pretrained weights and **keep training** some or all of them on your photos, with a small learning rate, so that the features themselves adapt. *Full* fine-tuning moves every weight; *partial* fine-tuning moves the last stages and keeps the first ones frozen (next section).

- The same two ideas under the names you will read: "logistic regression on fixed features" against "fine-tuned" (Kornblith et al., 2019); "linear probing (LP)" against "fine-tuning (FT)" (Kumar et al., 2022); "ConvNet as fixed feature extractor" against "finetuning the ConvNet" (the PyTorch tutorial).
- Both start from the same file and use the same loop. The difference is one question: **which parameters does the optimizer update?**

# The Mechanism as a Picture: Which Weights Move




- Six groups of weights (counts from the published file); the head is new in all three rows: `Linear(512, 5)` for the flowers.
- One switch per group, **frozen** or **trainable**. Thawing `layer4` alone already moves 75% of the weights: the last stage is the widest.

# The Math: One Update Rule, One Learning Rate per Group

The update, per group of parameters

$$\theta^l \leftarrow \theta^l - \eta^l \, g^l, \qquad g^l = \frac{\partial L}{\partial \theta^l}$$

$\theta^l$: the weights of group $l$ (a stage, or the head); $g^l$: their gradient of the loss (Day 16); $\eta^l$: **that group's** learning rate. Frozen group: $\eta^l = 0$, and `requires_grad = False` skips $g^l$ (Howard and Ruder, 2018).

With Adam (Day 17) the gradient is normalised, so one step moves a weight by **at most about** $\eta^l$, whatever the gradient's size. Rule: pretrained stages $10^{-5}$ to $10^{-4}$, head $10^{-3}$: **one rate per group**.

**With numbers from the weights file**

A typical weight in `layer4` of the published ResNet-18 has size 0.010 (mean $|w|$ of `layer4.1.conv2`). Five epochs of 108 batches: 540 steps.

- $\eta = 10^{-3}$: a step can change it by 0.001, **10% of its size**; 540 steps can rewrite it. The ImageNet knowledge is gone.
- $\eta = 10^{-5}$: 0.1% per step, 5% over the run: nudged, not rewritten.
- The new head starts random: nothing to lose, $10^{-3}$ is right *for it*.

# In Code: The Two Ways Side by Side

**Feature extraction** (Day 29):

```
model = resnet18(weights="IMAGENET1K_V1")
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(512, 5)
optimizer = torch.optim.Adam(
    model.fc.parameters(), lr=1e-3)
```

**Full fine-tuning** (today):

```
model = resnet18(weights="IMAGENET1K_V1")
# no freeze loop: every weight moves

model.fc = nn.Linear(512, 5)
optimizer = torch.optim.Adam(
    model.parameters(), lr=1e-4)  # small
```

- Two lines differ: the freeze loop, and which parameters the optimizer gets with which `lr`. The loop, the pipeline and the ImageNet `Normalize` are unchanged. (`"IMAGENET1K_V1"` is the tutorial's spelling of `ResNet18_Weights.DEFAULT`.)
- The PyTorch tutorial writes the same pair with `optim.SGD(..., lr=0.001, momentum=0.9)` and a `StepLR`; on ants against bees (244 training photos) both reach **94.8%**, the frozen one in 28 s, the fine-tuned one in 37 s.
- Fine-tuning computes the gradient through all 11 million weights: 2 to 3 times the time per epoch, and memory for every stage's activations.

# The Evidence: When Fine-Tuning Pays, and When It Does Not

| ResNet-50 on | /class | frozen | tuned | gain |
|---|---|---|---|---|
| Caltech-101 (objects) | 30 | 92.7 | 91.8 | −0.9 |
| Oxford Pets (37 breeds) | 99 | 91.5 | 92.5 | +1.0 |
| Oxford Flowers (102 kinds) | 20 | 93.2 | 97.5 | +4.3 |
| Food-101 (dishes) | 750 | 74.1 | 87.8 | +13.7 |
| FGVC Aircraft (100 types) | 67 | 58.5 | 86.6 | +28.1 |
| Stanford Cars (196 models) | 42 | 59.5 | 91.7 | +32.2 |

Test accuracy in %; photos per class from the dataset sizes. Kornblith, Shlens and Le (2019), Appendix Table I.1, "logistic regression" and "fine-tuned".

- **Pays most** when the classes are *fine-grained* (car models, aircraft types) or the photos far from ImageNet's kind: the encoder has to learn new features.
- **Pays least** when ImageNet already contains the kind: pets, everyday objects. There the probe is within a point of the best, at a third of the time.

## Common practice

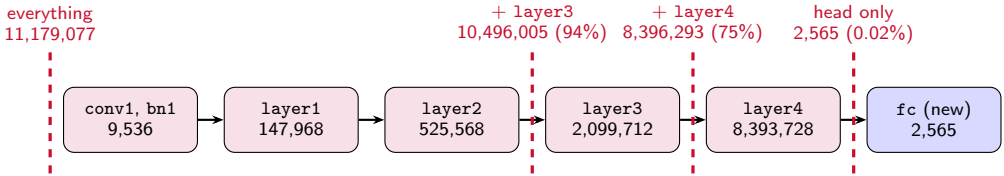
Probe first: train the head on the frozen encoder, read the validation accuracy. 93% leaves 4 points to win, 59% leaves 32. Then decide how much to unfreeze.

# Layer Freezing: **How Much of the Encoder Moves**

# Definition: A Freeze Line Through the Network

## Layer freezing

Keeping some stages of the pretrained network at their published values (`requires_grad = False`, not in the optimizer) while the stages above them train. Below the **freeze line**: a fixed feature extractor. Above it: fine-tuned.




frozen to the left of a line (ImageNet values kept), trained to the right; trainable counts above each line

- Four lines people use, four counts. Read yours from `sum(p.numel() for p in model.parameters() if p.requires_grad)`, every time you move the line.
- One stage further left adds that stage's gradient to every step: more memory, more time, more to overfit, more that can adapt.

# Why From the Top: What Yosinski et al. (2014) Measured

| Experiment | Result |
|---|---|
| copy layers 1 to 2, freeze, retrain the rest on the other half | transfers "almost perfectly": layers 1 and 2 are **general** (edges, colour blobs) |
| copy and freeze layers 3 to 5 | a drop: neighbouring layers were "co-adapted" and cannot be split |
| copy and freeze layers 6 to 7 | a drop because the features are **specific** to the first task |
| freeze all but the last layer | similar tasks lose 8%; dissimilar (man-made against natural) **25%** |
| copy, then **fine-tune** everything | +1.6% over training on the target alone |

AlexNet, 8 layers; ImageNet split into two halves of 500 classes, about 645,000 photos each.

Yosinski, Clune, Bengio and Lipson (2014), Figures 2 and 3, Table 1.

## The rule it gives

**Unfreeze from the top down.** The deepest stage is the most specific to ImageNet and the first to need retraining; the first stages are general and stay useful as they are. Howard and Ruder (2018) unfreeze "starting from the last layer as this contains the least general knowledge".

- The further your photos are from ImageNet (the 25% case), the more stages have to move: scans and satellite tiles are the far case, flowers the near one.

# In Code: Thaw `layer4`, Keep the Rest, Two Learning Rates

```python
model = resnet18(weights=ResNet18_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False                  # 1. freeze everything
for param in model.layer4.parameters():
    param.requires_grad = True                   # 2. thaw the last stage
model.fc = nn.Linear(model.fc.in_features, 5)    # 3. new head, trainable
optimizer = torch.optim.Adam([
    {"params": model.layer4.parameters(), "lr": 1e-4},    # pretrained: small
    {"params": model.fc.parameters(),     "lr": 1e-3},    # random start
])
print(sum(p.numel() for p in model.parameters() if p.requires_grad))   # 8396293
```

- The list of dictionaries is PyTorch's **per-parameter options**: each dictionary is a **parameter group** with its own `lr`; an option left out takes the optimizer's default. The documentation's own example gives `model.base.parameters()` `1e-2` and `model.classifier.parameters()` the default.
- `optimizer.param_groups` is that list; `optimizer.param_groups[0]["lr"]` reads or changes a group's rate during training (schedulers do exactly this). The gradient stops at `layer4`'s input, so a step costs less than full fine-tuning; the loop does not change.

# Discriminative Learning Rates: A Ladder of Step Sizes

### Definition (Howard and Ruder, 2018)

Give each layer its own learning rate, decreasing towards the input: choose the last layer's rate $\eta^L$ first, then $\eta^{l-1} = \eta^l/2.6$ for every layer below.

| Group | Rate | How |
|---|---|---|
| `fc` | $1.0 \times 10^{-3}$ | chosen |
| `layer4` | $3.8 \times 10^{-4}$ | $10^{-3}/2.6$ |
| `layer3` | $1.5 \times 10^{-4}$ | $/2.6$ again |
| `layer2` | $5.7 \times 10^{-5}$ | |
| `layer1` | $2.2 \times 10^{-5}$ | |
| `conv1` | $8.4 \times 10^{-6}$ | |

```
groups, lr = [], 1e-3
for part in [model.fc, model.layer4, model
    .layer3,
             model.layer2, model.layer1]:
    groups.append({"params": part.
        parameters(),
                   "lr": lr})
    lr /= 2.6                        # one rung
        down
optimizer = torch.optim.Adam(groups)
```

- The ladder says in numbers what the last frame said in words: the deeper (more general) the stage, the less it should move.
- **Measured** on their text classifier (validation error, lower is better): IMDb, one rate 6.87%, the ladder 5.57%; TREC-6 6.86% to 6.21% (Table 7). Now the fast.ai default for vision too.

# Gradual Unfreezing: One More Stage per Epoch

**Definition (Howard and Ruder, 2018)**

"Unfreeze the last layer and fine-tune all unfrozen layers for one epoch. Then unfreeze the next lower frozen layer and repeat."




purple: training, grey: frozen

```
optimizer = torch.optim.Adam(model.fc.parameters
    (), lr=1e-3)
thaw = {1: model.layer4, 2: model.layer3}   #
    epoch -> stage
for epoch in range(5):
    if epoch in thaw:
        for param in thaw[epoch].parameters():
            param.requires_grad = True
        optimizer.add_param_group(
            {"params": thaw[epoch].parameters(),
                "lr": 1e-4})
    train_one_epoch(model, train_loader,
        optimizer)
```

- `add_param_group` appends a group to a running optimizer: the stage moves from the next step on. `train_one_epoch` is Week 3's loop.
- **Measured** (Table 7, IMDb error): head only 6.49%, everything at once 6.87%, gradual 6.37%; on the small TREC-6 the head-only run "severely underfits": 16.09%.

# The Strategies, With Their Sources

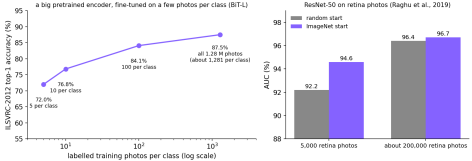
| Strategy | What moves | Learning rate | When | Source |
|---|---|---|---|---|
| linear probing (feature extraction) | the head | $10^{-3}$ | few photos per class, photos like ImageNet's, a short budget; always as the first probe | Kornblith et al. (2019); Kumar et al. (2022); PyTorch tutorial |
| last stage and head | `layer4`, `fc` | $10^{-4}$, $10^{-3}$ | hundreds per class, a few points left to win | Yosinski et al. (2014): the top layers are the specific ones |
| discriminative rates | every thawed stage, slower downwards | $\eta^{l-1} = \eta^{l}/2.6$ | whenever more than one stage moves | Howard and Ruder (2018) |
| gradual unfreezing | one more stage per epoch | the ladder | small data, forgetting risk | Howard and Ruder (2018); Felbo et al. (2017) |
| full fine-tuning | everything | Adam $10^{-5}$ to $10^{-4}$; SGD $10^{-3}$ | many photos, fine-grained classes, photos far from ImageNet | PyTorch tutorial; Kornblith et al. (2019); Kolesnikov et al. (2020) |
| LP-FT | the head first, then everything | $10^{-3}$, then $10^{-5}$ to $10^{-4}$ | the default when unsure: best in-distribution and under shift | Kumar et al. (2022) |
| surgical | one block chosen by the kind of shift | as above | corrupted inputs: first block; changed labels: last layer | Lee et al. (2023) |

- In a competition the order is fixed: **probe, read, thaw one stage, read again**. Each row is a move you can name in the write-up, with the paper that measured it.

# The Small-Data Regime **and Catastrophic Forgetting**

## Definition

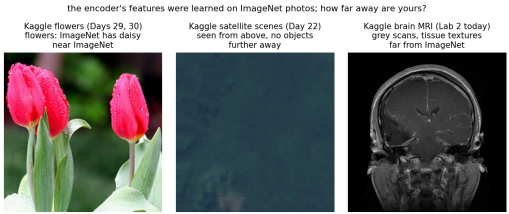
Tens to a few hundred labelled photos per class, against ImageNet's 1,281. What you *keep* from the pretrained file decides the score.




Sources: Kolesnikov et al. (2020), Section 3.5 (BiT-L, a ResNet-152x4 pretrained on 300 million images); Raghu et al. (2019), Tables 1 and 3; `scripts/day30_finetune_figures.py`.

- A strong pretrained encoder fine-tuned on **5 photos per class** reaches 72.0% on ImageNet's 1,000 classes; 100 per class 84.1%; all 87.5%. The first photos are worth the most.
- The pretrained start is worth 2.4 AUC points with 5,000 retina scans, 0.3 with 200,000: **the fewer the photos, the more the ImageNet weights carry**.

# How Far From ImageNet Are Your Photos?




Source: one photo each from Kaggle's *Flowers Recognition*, *Satellite Image Classification* and *Brain Tumor MRI* sets (the first file of a class folder); `scripts/day30_finetune_figures.py`.

- **Medical scans** (Raghu et al., 2019): on 200,000 retina and chest X-ray images, ImageNet weights "do not significantly help" the final score (ResNet-50: 96.4 against 96.7 AUC); the feature reuse that happens is "mostly restricted to the bottom two layers". Far from ImageNet, the probe is weak and the upper stages must move; the pretrained start still trains faster.
- **The kind of shift picks the block** (Lee et al., 2023): corrupted or differently lit inputs, tune the *first* block; changed labels, the *last* layer; new objects, the middle. Lab 2 tests the far case on brain scans.

# Common Practice: Photos per Class Against Distance

| Photos per class | Photos like ImageNet's | Photos far from ImageNet | Always |
|---|---|---|---|
| under 50 | head only, $10^{-3}$, 5 to 10 epochs | head, then `layer4` at $10^{-5}$ | augmentation, early stopping |
| 50 to 500 | `layer4` and head, $10^{-4}$ / $10^{-3}$ | `layer3` up, the ladder | probe first, read, then thaw |
| over 500 | LP-FT: probe, then everything at $10^{-5}$ to $10^{-4}$ | everything, $10^{-4}$, more epochs | keep the best epoch by validation |

Common practice as the sources state it: the PyTorch tutorial (120 per class, both ways 94.8%); Kornblith et al. (2019); Howard and Ruder (2018), the ladder and gradual unfreezing; Kumar et al. (2022), LP-FT; Kolesnikov et al. (2020), BiT-HyperRule: 500 steps, no MixUp under 20,000 examples, SGD at $3 \times 10^{-3}$, crops and flips; Raghu et al. (2019), medical.

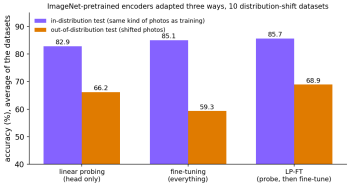
- The table is a **starting point**, not a law: every row ends with "read the validation accuracy". Your split decides, as in every week of this course.
- **Fewer photos → fewer weights move, smaller steps, fewer epochs, more augmentation.** More photos or a farther domain → move more, still with small steps.
- Time counts too: an epoch of full fine-tuning costs 2 to 3 head-only epochs. With 10 minutes the probe may be all you can afford.

# Catastrophic Forgetting

## Definition

A network trained on one task keeps training on a new one; the new updates overwrite the weights that held the old knowledge, and the old task's accuracy collapses (McCloskey and Cohen, 1989). In fine-tuning, the ImageNet features get **distorted**.

- **Measured** (Kumar et al., 2022, 10 datasets): fine-tuning beats linear probing on the training kind of photos, 85.1% against 82.9%, but *loses* on shifted photos, 59.3% against 66.2%.
- **How you notice it:** the training loss keeps falling while the accuracy on the old classes, or on slightly different photos, falls. IOAI 2026: naive training "will torch the old classes".




Source: Kumar, Raghunathan, Jones, Ma and Liang (2022), Tables 1 and 2, averages; `scripts/day30_finetune_figures.py`.

## What prevents it

Freeze more; smaller steps; fewer epochs; probe the head *before* unfreezing (LP-FT: 85.7% and 68.9%); old photos in the batches; the old model's answers as a target (the practice task).

# The Recipe That Keeps the Knowledge: Probe, Then Fine-Tune

```
model = resnet18(weights=ResNet18_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 5)
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
for epoch in range(3):                          # phase 1: linear probing, frozen features
    train_one_epoch(model, train_loader, optimizer); print(evaluate(model, val_loader))

for param in model.parameters():
    param.requires_grad = True                  # phase 2: thaw everything, a new optimizer
encoder = [p for n, p in model.named_parameters() if not n.startswith("fc")]
optimizer = torch.optim.Adam([{"params": encoder, "lr": 1e-5},
                              {"params": model.fc.parameters(), "lr": 1e-4}])
for epoch in range(3):                          # small steps: the head already points right
    train_one_epoch(model, train_loader, optimizer); print(evaluate(model, val_loader))
```

- Why it works (Kumar et al., 2022): with a random head the first gradients are large and random, and they land on the encoder; a probed head sends small, sensible gradients: the features move 10 to 100 times less.
- Read the validation accuracy after each phase: if phase 2 does not add, stop at the probe.

# Fine-Tuning: **Olympiad Practice**

# Practice Task: Fix the Weak Classes Without Breaking the Rest

## Task

You receive a trained image classifier as a weights file: a small vision transformer (`timm`'s TinyViT, 5 million parameters) with a head for **102 classes**. It "performs well on some classes, but struggles with others"; you get extra labelled photos for the **10 weak classes only**, and no photos of the other 92. The hidden test set has all 102 classes and the score is accuracy over all of them. The inference function is fixed (you hand in a `state_dict`). "Fine-tune the model so it handles all classes effectively, and make sure it doesn't forget what it already knows: catastrophic forgetting is your enemy."

- **Question 1:** the baseline notebook fine-tunes *everything* with AdamW at $3 \times 10^{-4}$ for 3 epochs and scores 0.53. In today's words, what is wrong with that recipe?
- **Question 2:** you have no photos of the 92 old classes. How can you still tell, during training, whether the model is forgetting them?
- **Question 3:** which parameters move, at what rate, and what do you hand in?

Source: NEOAI 2025 (Northern Eurasia), Task 2, "Underfitting CV" (`kaggle.com/competitions/neoai-2025-underfitting-cv`); quotes from the organisers' baseline notebook.

Scores from a published solution write-up (`github.com/stefanasandei/roai-solved`): baseline 0.53, the write-up's run 0.72, the best contestant 0.78.

# Solution Idea and Code

**Recognise:** fine-tuning a given model on few photos; forgetting is scored. Keep the 102-row head.

```
teacher = copy.deepcopy(model).eval()                          # the original weights,
    frozen
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)  # small steps for every
    weight
for xb, yb in train_loader:                                     # the 10 weak classes
    logits = model(xb)
    with torch.no_grad():
        old = teacher(xb).softmax(dim=1)                        # the old answers
    kd = F.kl_div(logits.log_softmax(dim=1), old, reduction="batchmean")
    loss = 0.7 * criterion(logits, yb) + 0.3 * kd               # fit the new, stay close to
        the old
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

1. $3 \times 10^{-4}$ on every weight of a trained model is the "rewrite it" step of the math frame: three epochs torched the 92 classes (0.53). Use $5 \times 10^{-5}$, clip gradients, label-preserving augmentation.
2. No old photos, so the old *model* is the reference: a frozen copy (the teacher) whose probabilities the new model stays close to, *knowledge distillation*. Watch held-out accuracy and agreement with the teacher.
3. Hand in `model.state_dict()`: 0.72 in the write-up against the baseline's 0.53.

# Fine-Tuning: **Limitations & What Comes Next**

# Limitations

- **Every number today is someone else's.** Kornblith's 32 points on cars, Kumar's 7-point drop under shift, the 2.6 ladder: measured on their data. Your split decides: "probe, read, thaw, read again", never "unfreeze because the slide said so".
- **Fine-tuning costs.** The gradient through 11 million weights takes 2 to 3 times the time and memory of the probe; ResNet-50 four times ResNet-18's. In a 10-minute budget, count epochs before choosing the freeze line.
- **More moving weights, more ways to overfit.** With tens of photos per class, full fine-tuning can land below the probe (Kumar et al., 2022); with thousands the probe "severely underfits" (Howard and Ruder, 2018). Neither extreme is the default.
- **Far from ImageNet, the file helps less.** On 200,000 scans the pretrained start changed the AUC by 0.3 (Raghu et al., 2019); it still converged faster. Expect to move more stages and a weak probe.

The quiet mistake: BatchNorm keeps moving

`requires_grad = False` freezes weights, not **BatchNorm's running mean and variance** (Day 17): in `model.train()` mode the 20 BatchNorm layers of a "frozen" ResNet-18 still update their 9,620 running numbers from your batches. The fix for a frozen encoder: after `model.train()`, set every `nn.BatchNorm2d` of the frozen stages to `eval()`.

# What Comes Next

- **Tomorrow (Day 31):** the encoder's 512 numbers as **embeddings**: distances and similarity between photos, nearest-neighbour search, and CLIP, which puts photos and sentences in one space: classification with no training at all.
- **Day 32, self-study:** today's choices step by step: feature extraction against fine-tuning on one dataset, the freeze line moved one stage at a time, the numbers read off the validation split.
- **Day 33, self-study:** embedding models, linear probing as a measurement, CLIP zero-shot and retrieval.
- **Day 34, Olympiad practice:** small-data fine-tuning habits: what wins points, what loses them. **Friday:** the Week 5 competition.
- **Today's labs:** Lab 1 trains the three strategies on the flowers on the same split (frozen head; `layer4` thawed at $10^{-4}$; everything at $10^{-5}$), with trainable counts, time per epoch, and how far each stage's weights moved. Lab 2 takes brain MRI scans, far from ImageNet: probe, read, fine-tune, read, confusion matrix, submission.

# Fine-Tuning: **Summary**

# Summary

- **Two ways: feature extraction** freezes the encoder and trains a head on its 512 numbers (**linear probing** when the head is one `Linear`: it measures the features); **fine-tuning** keeps training the pretrained weights with small steps so the features adapt. Same file, same loop; only the optimizer's parameters differ.
- **The math:** $\theta^l \leftarrow \theta^l - \eta^l g^l$, one learning rate per group. A `layer4` weight is about 0.010 in size: $\eta = 10^{-3}$ can rewrite it in a few hundred steps, $10^{-5}$ nudges it. Pretrained stages $10^{-5}$ to $10^{-4}$, the random head $10^{-3}$.
- **Layer freezing in code:** freeze all, thaw `model.layer4`, a new `fc`, an optimizer with a list of parameter groups (`{"params": ..., "lr": ...}`); the ladder $\eta^{l-1} = \eta^l / 2.6$; `add_param_group` for gradual unfreezing.
- **How much to unfreeze:** from the top down (Yosinski et al.: layers 1 and 2 general, the top specific; 8% against 25% drop for near against far tasks). Fine-tuning adds 1 point on pets, 32 on car models. Probe first, then decide.
- **Small data and forgetting:** fewer photos $\rightarrow$ fewer weights move, smaller steps, fewer epochs, more augmentation (BiT: 5 per class, 72%). Fine-tuning can distort the features (Kumar et al.: $+2$ in-distribution, $-7$ under shift); LP-FT, replay and a frozen teacher keep the knowledge.
- **In a task:** probe, read, thaw one stage, read again; keep a given head whose classes are scored; watch the old classes.

# Fine-Tuning: **References & Resources**

# References & Resources


[1] Yosinski, J., Clune, J., Bengio, Y. and Lipson, H. (2014). How transferable are features in deep neural networks? *NeurIPS* 27. AlexNet on two halves of ImageNet; Figures 2 and 3, Table 1: general first layers, specific top layers, the 8% against 25% drop, the $+1.6\%$ boost from fine-tuning.

[2] Kornblith, S., Shlens, J. and Le, Q. (2019). Do better ImageNet models transfer better? *CVPR*. Appendix Table I.1, ResNet-50 rows: logistic regression on fixed features, fine-tuned, from scratch, 12 datasets.

[3] Howard, J. and Ruder, S. (2018). Universal language model fine-tuning for text classification. *ACL*. Discriminative fine-tuning ($\eta^{l-1} = \eta^l/2.6$), gradual unfreezing, catastrophic forgetting; Table 7.

[4] Kumar, A., Raghunathan, A., Jones, R., Ma, T. and Liang, P. (2022). Fine-tuning can distort pretrained features and underperform out-of-distribution. *ICLR*. Linear probing, fine-tuning and LP-FT on 10 distribution-shift datasets; Tables 1 and 2.

[5] Raghu, M., Zhang, C., Kleinberg, J. and Bengio, S. (2019). Transfusion: understanding transfer learning for medical imaging. *NeurIPS* 32. Retina and CheXpert; Tables 1 to 3; feature reuse in the lowest layers.

[6] Kolesnikov, A., Beyer, L., Zhai, X., Puigcerver, J., Yung, J., Gelly, S. and Houlsby, N. (2020). Big Transfer (BiT): general visual representation learning. *ECCV*. Few-shot transfer (Section 3.5) and the BiT-HyperRule (Section 3.3).

# References & Resources (cont.)


[7] Lee, Y., Chen, A., Tajwar, F., Kumar, A., Yao, H., Liang, P. and Finn, C. (2023). Surgical fine-tuning improves adaptation to distribution shifts. *ICLR*. Which block to tune for input-level, feature-level and output-level shifts.

[8] McCloskey, M. and Cohen, N. (1989). Catastrophic interference in connectionist networks. *Psychology of Learning and Motivation* 24. Felbo, B. et al. (2017). Using millions of emoji occurrences to learn any-domain representations. *EMNLP*. Chain-thaw.

[9] PyTorch tutorials, *Transfer learning for computer vision*: "Finetuning the ConvNet" and "ConvNet as fixed feature extractor", ants against bees, best validation accuracy 0.9477 in 37 s and 28 s. https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

[10] PyTorch documentation, `torch.optim`: "Per-parameter options" (parameter groups, `model.base` at `1e-2`, `model.classifier` at the default `1e-3`), `add_param_group`. https://docs.pytorch.org/docs/stable/optim.html

[11] NEOAI 2025 (Northern Eurasia), Task 2, "Underfitting CV", the practice task: a TinyViT with a 102-class head, extra photos for 10 classes, fixed inference. https://www.kaggle.com/competitions/neoai-2025-underfitting-cv Solution write-up and scores: https://github.com/stefanasandei/roai-solved IOAI 2026, At-Home Round, Task 1 (the "torch the old classes" warning). https://github.com/IOAI-official/IOAI-2026

# References & Resources (cont.)

[12] Kaggle datasets through `kagglehub`: *Flowers Recognition* (`alxmamaev/flowers-recognition`, Lab 1), *Brain Tumor MRI Dataset* (`masoudnickparvar/brain-tumor-mri-dataset`, Lab 2, 7,200 scans, four classes), *Satellite Image Classification* (`mahmoudreda55/satellite-image-classification`). Parameter counts and weight sizes read from `ResNet18_Weights.DEFAULT`. Figures: `slides-builder/scripts/day30_finetune_figures.py`; no model was trained for this deck, every comparison number is quoted from the sources above.